

Page 1 — Python Fundamentals

Knowledge Base Document: Python | Page 1 of 5

Python is a high-level, general-purpose programming language widely used in data science, automation, web development, scientific computing, and artificial intelligence. Its syntax emphasizes readability, which makes Python suitable for both beginners and experienced developers.

Variables in Python are dynamically typed. A variable is created when a value is assigned to it, for example, `x = 10`. Common built-in data types include `int`, `float`, `str`, `bool`, `list`, `tuple`, `set`, and `dict`. The `type()` function can be used to inspect the type of an object.

Python uses indentation to define code blocks instead of braces. Conditional statements include `if`, `elif`, and `else`. Iteration is commonly performed with `for` and `while` loops. The `range()` function is frequently used to generate integer sequences for loops.

Functions are reusable blocks of code defined with the `def` keyword. Functions can accept positional arguments, keyword arguments, default values, and variable-length arguments. A `return` statement sends a result back to the caller.

Exception handling uses `try`, `except`, `else`, and `finally`. It is preferable to catch specific exceptions instead of using a broad `except` clause because specific handling makes failures easier to diagnose.

Page 2 — Python Data Structures and Functions

Knowledge Base Document: Python | Page 2 of 5

Lists are ordered and mutable collections. They support indexing, slicing, `append()`, `extend()`, `insert()`, `remove()`, `pop()`, and sorting operations. Tuples are ordered but immutable, making them useful for fixed collections of values.

Dictionaries store key-value pairs and provide efficient lookup by key. Sets store unique elements and support operations such as union, intersection, and difference. Choosing the correct data structure can substantially improve code clarity and performance.

List comprehensions provide a compact way to construct lists. Dictionary and set comprehensions provide similar functionality. They should be used when they improve readability; complicated comprehensions are often better expressed with ordinary loops.

Lambda expressions create small anonymous functions. Functions such as `map()`, `filter()`, and `sorted()` can accept callable objects. In production code, ordinary named functions are often preferable when the transformation is complex or reused.

Modules organize Python code into reusable files. Packages group related modules. Virtual environments isolate dependencies between projects. A `requirements.txt` file can record project dependencies so that an environment can be reproduced.

Page 3 — Object-Oriented Python

Knowledge Base Document: Python | Page 3 of 5

Object-oriented programming organizes behavior and data into classes and objects. A class defines attributes and methods, while an object is an instance of that class. The `__init__` method is commonly used to initialize instance attributes.

Encapsulation groups implementation details with the operations that manipulate them. In Python, naming conventions such as a leading underscore communicate that an attribute or method is intended for internal use.

Inheritance allows a class to reuse or specialize behavior from another class. Method overriding allows a child class to provide its own implementation. Composition is an alternative design in which objects contain other objects rather than inheriting from them.

Polymorphism allows different objects to respond to the same operation according to their own implementation. Python's duck typing often enables polymorphism without requiring an explicit inheritance relationship.

Dataclasses can reduce boilerplate for classes that primarily store data. Special methods such as `__str__`, `__repr__`, `__len__`, and `__eq__` can customize object behavior. Good class design should favor simple interfaces and clear responsibilities.

Page 4 — Python for Data Science

Knowledge Base Document: Python | Page 4 of 5

NumPy provides efficient multidimensional arrays and vectorized numerical operations. Pandas provides Series and DataFrame structures for tabular data manipulation. Matplotlib and related libraries are commonly used for visualization.

A typical data-science workflow loads data, inspects schema and missing values, cleans inconsistent records, transforms features, explores distributions, and prepares data for modeling. Reproducible preprocessing is important because training and inference must apply compatible transformations.

Pandas operations include filtering rows, selecting columns, groupby aggregation, merging datasets, reshaping tables, handling missing values, and converting data types. Vectorized operations are generally preferred over Python-level row loops for performance.

Python environments should be isolated for data-science projects because numerical libraries often have strict version compatibility requirements. Pinning versions is useful when reproducibility matters.

Testing is important even in data-science code. Assertions, unit tests, validation checks, and deterministic random seeds can detect unexpected changes in input data or preprocessing logic.

Page 5 — Python Best Practices

Knowledge Base Document: Python | Page 5 of 5

Readable Python follows the principle that code is read more often than it is written. Descriptive names, small functions, consistent formatting, and limited nesting improve maintainability.

PEP 8 provides widely adopted style recommendations. Type hints can document expected interfaces and improve editor support. Docstrings should explain the purpose, parameters, return values, and important assumptions of reusable functions.

Logging is preferable to scattered print statements in production applications. Logs should provide useful context without exposing secrets or sensitive information. Configuration such as API keys should be supplied through environment variables or secure secret management.

Performance optimization should begin with measurement. Profiling can identify actual bottlenecks. Vectorization, appropriate data structures, caching, and algorithmic improvements generally provide more value than premature micro-optimization.

A robust Python project separates configuration, business logic, data access, and presentation where appropriate. Version control, automated tests, dependency management, and clear documentation make the project easier to deploy and maintain.